1 Evaluation Methodology

This chapter details the methodology used to evaluate the **Agentic Project Management (APM)** framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants.

The evaluation was designed to measure three distinct dimensions:

- **Economic Efficiency** *[Cost framing pending literature review.]*
- **Process Efficiency** (human cognitive load and interaction burden)
- **Functional Efficacy** (ability to complete non-trivial tasks)

[Overview of the Copilot-based comparative study — forthcoming.]

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM (i.e., the Planning and Implementation phases) yields a tangible return on investment compared to a single-agent workflow.

We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):** *[Cost framing pending literature review — to be reframed around request consumption and resource efficiency rather than raw token cost.]*
- **RQ2 (Process Efficiency):**
  - Does the structured coordination of APM reduce the "human burden" during execution? Specifically, does it reduce the need for user interventions, corrections, and clarifications?
  - Is the reduction in human burden due to the structured coordination of APM significant enough to justify the structural overhead?
  - Are the agents able to effectively coordinate and complete the task without the need for constant human oversight?
- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where unstructured agentic workflows typically encounter "Agentic Handoff Failures"?
  - How does the performance of APM compare to the performance of a single-agent workflow?

1.2 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`, a Python-based Model Context Protocol (MCP) server designed to interface with the `openeclass` platform. Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

1.2.1 The Task:

The final task requires participants to implement a new `get_course_announcements` tool in the Python MCP server and, crucially, to create a complete native C replica of the MCP server. The task specification was refined through iteration to ensure sufficient complexity for differentiating multi-agent from single-agent approaches.[1]

This C implementation adds significant complexity, turning the assignment into a challenge of managing code translation, memory safety, and protocol details across two very different languages. As such, the task is specifically designed to stress-test the limits of agent coordination and reveal differences between multi-agent and single-agent approaches, especially in scenarios where context management becomes a bottleneck.

Key Technical Challenges:

1. **Dynamic Data Source:** The required data for implementing the new feature is not available in static HTML for parsing. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.
2. **Nested Parsing:** The server returns JSON where the payload is an array of raw HTML strings, requiring a second layer of parsing to extract the data.
3. **Client-Side Filtering:** The legacy API does not support date filtering. The agent must implement robust client-side logic to parse and filter unique locale-formatted dates.
4. **C Implementation:** Agents must build a native C MCP server replica, manually implementing JSON-RPC 2.0 with standard C string manipulation (no external JSON libraries such as cJSON/Jansson). Use of `libcurl` for HTTP/HTTPS and `libxml2` for HTML parsing, was required for the eclass MCP server functionality.
5. **Memory Safety:** The C implementation must be memory-safe, passing Valgrind checks without leaks or errors.

Deliverables:

1. Core server logic in modular Python files (existing and new functionality).
2. A native C implementation of the MCP server with equivalent (existing and new) functionality.
3. A working build system (Makefile) for the C replica.

[1] Task specification and grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task

1.2.2 Grading:

Submissions are evaluated using an automated black-box test suite with comprehensive validation, grading, metric extraction, and dataset aggregation capabilities (see the evaluation appendix for detailed infrastructure documentation).[2]

The test suite awards **48 points total**: 12 points for baseline functionality (existing Python tools), 12 points for the new Python feature (`get_course_announcements`), and 24 points for the complete C replica implementation. A submission with no patch applied receives the baseline score of **12/48 points**.

All tests are run against fixed baseline commits to ensure reproducibility: `eclass-mcp-server` at commit `8b4e39c` and `openeclass` at commit `e8b3329` (Docker image `openeclass:test`).

1.2.3 Repository Architecture and Data Management

The case study employs a two-repository system separating public resources from sensitive participant data. The **task repository** (`bsc-apm-case-study-task`) is openly accessible and includes task specifications, grading infrastructure, and participant guides. The **dataset repository** (`bsc-apm-case-study-data`) contains all submitted participant data: code patches, sanitized chat transcripts (see Section 1.2.4), and anonymized session metadata.

Access to the dataset repository's secret access key requires explicit authorization and compliance with data usage agreements. This controlled access mechanism enables peer review and reproducibility while maintaining participant privacy protections.

1.2.4 Sanitization and PII Handling

Chat exports contain sensitive information (unstructured user-input text, file paths, command histories, timestamps). To protect participant privacy while enabling reproducibility, all exports are sanitized using reversible encryption before storage.

Each chat is assigned a unique identifier via SHA256 hashing of concatenated user messages for anonymous cataloging. Essential metadata (chat ID, model IDs, request count) is preserved in plaintext to enable dataset organization without decryption, while conversation content is encrypted using an HMAC-SHA256 XOR cipher. The same sanitization method is applied uniformly across both platforms.

For detailed technical specifications of the anonymization and encryption processes, see the evaluation appendix.

[2] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

1.3 Study Design

The study is conducted on a single IDE platform, **GitHub Copilot**, and compares the two frameworks under evaluation across a homogeneous cohort of participants. It is designed as a within-subjects experiment, so that each participant contributes data points for both frameworks and per-participant variance in skill, prior experience, and working style is absorbed into each pairwise comparison rather than confounding it.

1.3.1 Experimental Design

Each participant completes the case study task twice: once with APM, once with Spec-kit. The order of the two frameworks is **counterbalanced** across participants, so that approximately half of the cohort encounters APM first and the other half encounters Spec-kit first. This counterbalancing lets us attribute any observed S1-to-S2 improvement to task familiarity rather than to the framework itself, and is analyzed explicitly in Section 2 under Learning Effects.

The two sessions are isolated from one another via dedicated Git branches created from the baseline commit, so no code, state, or context from the first session can influence the second. The branch workflow, patch-generation steps, and submission format are documented in the evaluation appendix.

1.3.2 Data Source

The primary data source for the study is **GitHub Copilot chat exports**, captured in JSON form from the VS Code Command Palette at the end of each session. Copilot's export format provides structured records of every conversation, including model identifiers, message timestamps, tool invocations, and per-message speaker roles. This granularity enables programmatic extraction of both request-level and interaction-level metrics directly from the exports, without any manual annotation of the transcripts.

APM's multi-agent workflow spans multiple Copilot conversations within a single session — one for the Planner, one for the Manager, and one per Worker defined in the Plan — while Spec-kit operates within a single conversation. To keep the two frameworks comparable, all metrics are computed by aggregating every conversation file belonging to a participant submission into a unified session-level view, so that the total interaction cost of the session is captured regardless of how the work was distributed across conversations.

1.3.3 Participant Pool

Participants are drawn from a cohort of University students and working professionals with active access to GitHub Copilot, either through the GitHub Education Pack or through a standard premium subscription. Access requirements, recruitment criteria, and the participant-facing workflow are documented in the participant guide (see the evaluation appendix).

1.4 Frameworks Under Evaluation

Two spec-driven workflow configurations are evaluated. Both are open-source and both run on the same underlying IDE platform; the key axis of comparison is whether spec-driven execution is distributed across a team of specialized agents (APM) or handled by a single agent following a structured template (Spec-kit).

1. **Agentic Project Management (APM v1):** The proposed framework. APM replaces the single continuously-growing chat with a team of specialized agents coordinating through file-based artifacts, and structures work into two phases:

   - *Planning Phase:* A **Planner** conducts structured discovery with the User and decomposes the project into three planning documents — a *Spec* (what to build: design decisions and workspace layout), a *Plan* (how work is organized: Stages, Tasks, Worker assignments, dependencies), and *Rules* (how work is performed: universal execution patterns written to the platform's rules file). The Planner runs once at project start and hands off to execution.
   - *Implementation Phase:* A **Manager** dispatches Tasks to **Workers**, reviews the resulting Task Logs, and drives follow-ups, planning-document revisions, or the next ready dispatch. Workers execute Tasks against explicit validation criteria and write structured logs that the Manager reads to coordinate progress.

   All agent coordination is mediated through a file-based **Message Bus** — the Manager writes Task assignments, Workers write results back, and the User carries messages between conversations. When an agent's context fills, a **Handoff** procedure transfers working knowledge to a fresh instance so state survives in files rather than depending on any single conversation.

2. **GitHub Spec-kit:** A structured, spec-driven baseline. It utilizes a single-agent workflow guided by rigid templates and three sequential commands (`specify`, `plan`, `tasks`). All planning and implementation happens within one conversation, with the agent acting as both planner and executor.

1.5 Metrics and Data Collection

The metrics collected for this study are organized into five categories: session metadata, the three efficiency dimensions introduced above (economic, process, and reliability), and the functional grading rubric. Their concrete definitions are deferred to the forthcoming literature review on enterprise request-based cost framing, since the cost framing drives several of the metric-level design choices and a subset of the process and reliability metrics may be adjusted once it is established.

1.5.1 Session Metadata

[Forthcoming. Will cover the distinct model identifiers used across a session, the per-conversation and project-level timestamps, and the session count for each APM submission.]

1.5.2 Cost Metrics

[Forthcoming, blocked on the literature review. Will operationalise Copilot request consumption as a proxy for enterprise budget impact, with request-per-functional-point as the primary efficiency ratio.]

1.5.3 Reliability Metrics

[Forthcoming. Will cover platform-agnostic measures of system competence — tool success rate, tool retry rate, and user edit rate on terminal commands — as indicators of how autonomously each framework executes work.]

1.5.4 Process Metrics

[Forthcoming. Will cover behavior distribution (ratios of short, protocol, and standard user inputs), behavior magnitude (average characters per input bucket), and session duration, as indicators of the human coordination burden each framework imposes.]

1.5.5 Functional Metrics

The functional dimension is evaluated against the 48-point grading rubric already introduced in Section 1.2.2. Final code artifacts (`.patch` files) are scored across three dimensions:

- **Baseline Regression:** Ensures that existing functionality (12 points) remains intact after the patch is applied, isolating the framework's impact on code stability.
- **Functional Correctness:** End-to-end validation of the new Python feature (12 points) and the complete C replica (24 points) against the specification.
- **Robustness:** Edge-case handling including authentication failures, invalid inputs, empty responses, and memory safety (Valgrind validation for the C implementation).

2 Results

2.1 Functional Performance Analysis

Primary Metrics: Total score distribution, completion rates (48/48, 24-47, 12-23, baseline-only), baseline regression rates (python_baseline_score < 12), feature implementation success (Python feature and C replica completion rates).
Category Analysis: Success rates per test category (build, protocol, login, logout, courses, authstatus, announcements) for both Python and C implementations. Identifies framework-specific failure patterns and strengths.
Comparative Analysis: T-tests comparing APM vs. Spec-kit for total score, category scores, and completion rates. Effect sizes (Cohen's d) to assess practical significance.

2.2 Economic and Process Efficiency Analysis

[Forthcoming. Will reference the request-based cost framing and the redesigned process metrics once the literature review lands.]

2.3 Learning Effects and Confounding

Critical Issue: Participants complete the same task twice (Session 1 and Session 2) using different frameworks. This creates a learning effect where task familiarity from S1 influences S2 performance, potentially confounding framework comparisons.
Analysis Strategy:

- **Learning Effect Matrix:** Calculate mean improvement (percentage change) from S1 to S2 for both S1→S2 framework combinations across all metrics (functional scores, cost, process efficiency)
- **Order Effect:** Compare APM→Spec-kit vs. Spec-kit→APM directions. Tests whether order of framework exposure affects learning.
- **Baseline Learning:** Overall mean improvement across both directions (pure task learning effect regardless of framework)

Key Questions: Does task learning confound framework comparisons? Is there a "best first framework" that maximizes S2 improvement? How do we control for learning effects when comparing frameworks?

3 Discussion

3.1 Cost-Benefit Analysis: Context Overhead vs. Failure Prevention

[Forthcoming. To be reframed around request-consumption overhead versus failure prevention, once the literature review establishes the cost framing.]

3.2 Process Efficiency: Protocol Overhead vs. Intervention Reduction

[Forthcoming. To be updated for APM v1 terminology and the two-framework comparison.]

3.3 Functional Efficacy: Reliability and Failure Patterns

[Forthcoming. To be updated for the two-framework comparison.]

3.4 Learning Effects: Confounding and Framework Transferability

[Forthcoming. Simplified for the two-framework design (APM v1 versus Spec-kit, in both orders).]

3.5 Threats to Validity

[Forthcoming. To enumerate the threats relevant to a single-platform, two-framework study.]